

Universidad Tecnológica Nacional

Proyecto Final — Vitalys App

Gestión integral de un centro de salud híbrido:
gimnasio y consultorios de Nutrición, Psicología y Kinesiología

Carrera: Tecnicatura Universitaria en Programación a Distancia
Materia: Proyecto Final

Proyecto: Vitalys App
Grupo N.º: 181
Integrante: Basualdo Arcati, Pablo Gabriel Ramón
Integrante: Calcatelli, Renzo
Tutora: Sofía Raia
Año: 2026

Índice

Resumen y enlaces del proyecto
1. Problema y contexto
2. Objetivos y alcance
3. Análisis y modelo del dominio
4. Arquitectura y tecnologías
5. Implementación
6. Pruebas y validación
7. Resultados y conclusiones
Anexo. Documentación de referencia en el repositorio

Resumen y enlaces del proyecto

Vitalys App es una aplicación web responsive para gestionar un centro que combina un **gimnasio** con **consultorios** de Nutrición, Psicología y Kinesiología. Unifica en una sola plataforma la identidad de socios y pacientes, la agenda de turnos de ambos servicios, el registro de pagos con control de morosidad y las notificaciones por correo electrónico. Está construida con una API REST en Spring Boot (Java 17), un frontend en React con TypeScript y una base de datos PostgreSQL, desplegados en la nube sobre Render, Vercel y Supabase.

El sistema implementa los seis módulos comprometidos en la propuesta (RFC-001) y agrega, como cambios de alcance registrados, el panel de inicio del administrador ("Hoy en el centro"), los avisos de deuda y la función "Operar como", que permite a Recepción atender a un socio en el mostrador con una sesión acotada y auditada. Las reglas críticas del dominio (no superposición de turnos, cupo del gimnasio, morosidad, unicidad de pagos) se garantizan en la base de datos y en la capa de servicio, y están cubiertas por pruebas automatizadas que se ejecutan en integración continua con un umbral mínimo de cobertura del 90 %.

Recurso	Enlace
Aplicación (frontend)	https://vitalys-app-ayk2.vercel.app
API (backend)	https://vitalys-backend-039u.onrender.com
Documentación de la API (Swagger UI)	https://vitalys-backend-039u.onrender.com/swagger-ui/index.html
Repositorio	https://github.com/rcalcatelli/vitalys-app
Video final	https://youtu.be/LeFzBQFSr7M

Acceso para la corrección. La pantalla de ingreso muestra las cuentas de demostración de los tres roles (administración, profesional y socio/paciente); las contraseñas de prueba están documentadas en `db/dml/seed.sql`. El plan gratuito de Render suspende el backend tras un período sin tráfico, por lo que la primera solicitud puede demorar cerca de un minuto mientras el servicio arranca.

1. Problema y contexto

1.1. Situación que dio origen al proyecto

Los centros que combinan actividad deportiva con prestaciones de salud tienen dos servicios bajo el mismo techo cuyos datos, en general, no se comunican. En los establecimientos chicos esto toma la forma de herramientas desconectadas: planillas de socios, agendas en papel o por WhatsApp para cada profesional y cobros anotados en un cuaderno o en Excel. En los establecimientos grandes, que sí cuentan con un sistema propio, el problema no desaparece sino que cambia de forma: el gimnasio y la prestación de salud conviven en la misma plataforma, pero sus datos no se cruzan.

Las consecuencias son concretas: personas registradas dos veces (como socio y como paciente) con datos inconsistentes, turnos superpuestos o perdidos, pagos sin trazabilidad, deudas que nadie detecta a tiempo y una recepción saturada de tareas manuales y consultas telefónicas. Para el negocio esto se traduce en ingresos no cobrados y en decisiones tomadas sin información confiable.

1.2. Usuarios y actores involucrados

Actor	Rol en el proceso actual	Necesidad principal
Socio / paciente	Asiste al gimnasio, a consultorios o a ambos; muchas veces figura como dos personas distintas.	Reservar y cancelar turnos sin depender del horario de recepción y conocer su estado de cuenta.
Profesional (Nutrición, Psicología, Kinesiología)	Administra su agenda de forma aislada.	Ver su agenda del día, cargar su disponibilidad y registrar asistencia sin superposiciones.
Recepción / administración	Centraliza cobros, turnos y consultas; es el cuello de botella operativo.	Reducir tareas repetitivas y errores de registro; atender en el mostrador a quien no usa la aplicación.
Dueño / encargado	Supervisa ingresos y actividad.	Información consolidada del día y de los socios con deuda. En el MVP opera con el rol de administración.

1.3. Evidencia utilizada para validar el problema

A pedido de la tutora en la revisión de la primera entrega, el equipo trabajó con dos fuentes de evidencia complementarias, con distinto peso metodológico:

1. **Relevamiento de campo en un establecimiento real.** El 28/09/2026 se realizó una entrevista presencial a la persona encargada de la coordinación del **Centro Deportivo Jerárquicos**, siguiendo una guía de entrevista escrita de antemano. El registro completo, que distingue lo efectivamente respondido de lo que quedó pendiente, está en `docs/01-propuesta/relevamiento/entrevista-jerarquicos.md`. El establecimiento autorizó verbalmente que se lo nombre.
2. **Perfiles de usuario sintéticos.** Doce perfiles elaborados por el equipo (cuatro por rol) para contrastar el catálogo de requerimientos contra necesidades concretas y detectar huecos funcionales. Están rotulados explícitamente como sintéticos (`perfiles-usuario.json`) y no se presentan como testimonio de personas reales.

El relevamiento mostró que el establecimiento usa un sistema desarrollado a medida, con acceso web y móvil, en el que el socio reserva el gimnasio y, con la misma cuenta, pide turnos de consultorio. Aun así, lo que el propio centro señala como pendiente es **integrar los datos del gimnasio con los de la**

prestación de salud. Este hallazgo es relevante porque indica que el problema no se resuelve solo con tener software: persiste incluso en un establecimiento con sistema propio.

Más allá de confirmar el problema, el relevamiento obligó a cambiar reglas que el equipo había definido sin evidencia. Los hallazgos más importantes y su efecto sobre el diseño fueron:

#	Hallazgo	Efecto en Vitalys
7	El centro tolera hasta **6 meses de deuda** antes de suspender; mientras tanto el socio sigue ingresando y recibe avisos.	Se reescribió la regla de morosidad (RN-01) con un umbral configurable y un estado intermedio "con deuda" que no bloquea (migración V6).
11	Cancelar **libera el lugar** para otra persona; solo lo retiene cancelar con la franja ya iniciada.	RN-02 y RN-03 pasaron a definir la ocupación por el momento de la cancelación, con una única función compartida por los controles de la base (V8).
10 / 5	El profesional informa ausencias por un período; no estaba definido qué pasa con los turnos ya tomados.	Cancelación en cascada con vista previa del impacto, confirmación y aviso a cada paciente (RN-24).
4	Faltaba la fecha de inicio de la membresía: un socio nuevo podía aparecer como moroso desde el primer día.	Columna `fecha_inicio_membresia` (V4).
2 / 3	No se podía consultar el cupo de una franja de gimnasio ni estaba definido quién registra la asistencia.	Grilla horaria con cupo por franja y check-in a cargo de Recepción (Decisión de dominio 11).
8 / 9	El cobro real es por débito automático y el turno de gimnasio incluye la actividad.	Registrados como divergencias conscientes respecto del alcance del MVP (ver sección 2.4).

El relevamiento también confirmó decisiones previas: la identidad única socio-paciente sobre una sola cuenta, que la suspensión por deuda del gimnasio **no alcance a la prestación de salud**, que el profesional se limite a registrar la asistencia sobre su agenda y que el aviso de deuda deba preceder al bloqueo.

1.4. Necesidad concreta que se buscó resolver

Contar con una única plataforma, diseñada para el modelo híbrido, en la que cada persona exista una sola vez con su historial de turnos y pagos; en la que el sistema, y no la memoria de un operador, impida las superposiciones y condicione la reserva del gimnasio al estado de cuenta; y en la que el socio pueda autogestionar sus turnos las 24 horas, descomprimiendo a la recepción sin reemplazarla.

2. Objetivos y alcance

2.1. Objetivo general

Desarrollar y desplegar en la nube una aplicación web que centralice la gestión de personas, profesionales, turnos de gimnasio y de consultorio, pagos y notificaciones de un centro de salud híbrido, haciendo cumplir de forma automática las reglas de negocio del dominio.

2.2. Objetivos específicos

- Modelar una identidad única socio-paciente con baja lógica y trazabilidad de cada operación.
- Garantizar, a nivel de base de datos, que no existan turnos superpuestos ni franjas de gimnasio por encima de su cupo, incluso ante reservas simultáneas.

- Calcular la morosidad del gimnasio de forma acumulativa y con umbral de tolerancia, y aplicarla a la reserva sin afectar los turnos de consultorio.
- Registrar pagos de cuotas y sesiones con su medio de pago, y exponer el estado de cuenta al socio y a la administración.
- Notificar por correo las reservas, cancelaciones, recordatorios y deudas, sin que una falla del proveedor afecte la operación.
- Sostener el desarrollo con un proceso verificable: ramas y Pull Requests revisados, integración continua y un umbral de cobertura de pruebas.

2.3. Alcance finalmente implementado

Módulo	Funcionalidad implementada	Estado
1. Autenticación y roles	Registro público con rol forzado a socio/paciente; login por email o DNI; JWT; tres roles (ADMIN, PROFESIONAL, SOCIO_PACIENTE) con control de acceso en la capa de servicio.	Completo
1b. Operar como	Recepción opera en nombre de un socio con una credencial de 30 minutos, auditada y visible para el socio.	Completo (cambio de alcance)
2. Personas	Alta, modificación, baja lógica y reactivación; vínculo de la ficha con un usuario ya registrado; DNI único; columna de cuota y filtro de deuda.	Completo
3. Profesionales y disponibilidad	ABM con baja lógica en dos pasos; franjas semanales sin solapamiento; cancelación en cascada con vista previa del impacto al reducir la disponibilidad.	Completo
4. Agenda de turnos	Gimnasio: grilla, cupo por franja, un turno por día, feriados oficiales, anticipación de 1 hora a 7 días, check-in y ausentes automáticos. Consultorio: turnos según la disponibilidad del profesional y agenda del día. Cancelación con estado según la anticipación. Excepciones de morosidad autorizadas por el ADMIN.	Completo
5. Pagos	Cuotas mensuales y sesiones de consultorio con medio de pago; estado de cuenta (pagado, impago, pendiente) con 10 días de gracia; regla de morosidad compartida con la reserva.	Completo
6. Notificaciones	Confirmación, cancelación, recordatorio 24 h antes y avisos de deuda y suspensión, por la API de Brevo, con un reintento y alerta en el panel del ADMIN si falla.	Completo
Panel "Hoy en el centro"	Inicio del ADMIN: quién está en el gimnasio, turnos y consultas del día, ocupación por franja, socios que requieren atención y notificaciones fallidas.	Completo (cambio de alcance)

Además, la interfaz sigue el prototipo de diseño elaborado para el proyecto: shell común con barra lateral en escritorio y barra de pestañas en el celular, modo oscuro persistente, pantalla de inicio del socio y pantalla de Ingreso al gimnasio. Los desvíos deliberados respecto del prototipo están documentados con su motivo en `docs/02-diseno/wireframes.md`.

Todo cambio de alcance se registró en el RFC-001 mediante Pull Request, como se había acordado con la tutora. "Operar como" quedó asentado como **Decisión de dominio 12**, con su fundamento y sus límites (ver sección 4.5).

2.4. Funcionalidades que quedaron fuera y motivo del recorte

Funcionalidad	Motivo
Ficha clínica digital	Implica almacenar datos de salud (Ley 25.326) con exigencias de resguardo, retención y acceso que exceden un trabajo académico. Por minimización, el sistema no guarda diagnósticos ni motivos de consulta.
Pasarela de pago y débito automático	El relevamiento mostró que el cobro real es automático (hallazgo 8), pero integrar un procesador de pagos es un módulo en sí mismo. Los cobros se registran a mano con su medio, y el tipo `medio_pago` admite sumar Mercado Pago con una migración propia.
Pagos parciales, packs y planes	Introducen saldos, imputación y estados intermedios de deuda; el objetivo del MVP es la trazabilidad del pago, no una cuenta corriente (Decisión 6).
Actividad dentro del turno de gimnasio y clases grupales	El turno de gimnasio modela una franja con cupo (hallazgo 9). Agregar actividades exige otro modelo de cupos.
Reportes de ingresos y ocupación	Estaban como "nice to have" y se descartaron para priorizar los seis módulos. El panel "Hoy en el centro" cubre la información operativa del día.
Recuperación de contraseña por email	Ningún requerimiento la contemplaba. La pantalla existe y lo informa con claridad; la contraseña se restablece en el centro.
Varios roles por usuario	Un profesional que además es socio necesita dos cuentas. Es una limitación de diseño documentada (Decisión 8): el caso no apareció como frecuente.
Aplicaciones nativas	La aplicación web es responsive y cubre el uso desde el celular.
Listados generales de pagos	Los endpoints `GET /api/pagos` y `GET /api/pagos/{id}` se difirieron: el estado de cuenta por persona cubre la consulta necesaria.

3. Análisis y modelo del dominio

3.1. Principales entidades y relaciones

Entidad (tabla)	Responsabilidad
`usuarios`	Credenciales (email y hash bcrypt), rol único y estado habilitado.
`personas`	Identidad única del socio/paciente: DNI único, estado ACTIVO/INACTIVO con fecha de baja, si es socio del gimnasio y desde cuándo.
`profesionales`	Especialidad, duración del turno por profesional y baja lógica.
`disponibilidad_profesional`	Franjas semanales (día, hora de inicio y de fin) de cada profesional.
`turnos`	Reservas de gimnasio y de consultorio en una sola tabla, con tipo, inicio, fin, estado, quién reservó, quién y cuándo canceló, y hora real del check-in.
`pagos`	Cuota mensual (con período) o sesión de consultorio (con turno), monto, medio de pago y quién lo registró.
`excepciones_morosidad`	Autorizaciones puntuales del ADMIN para que un socio suspendido pueda reservar el gimnasio.
`notificaciones`	Registro de cada correo: tipo, destinatario, resultado y, en los avisos de deuda, el período avisado.
`feriados` y `configuracion_gym`	Días en que el gimnasio no abre (sincronizados con el calendario oficial) y parámetros: cupo por franja y meses de tolerancia de morosidad.
`auditoria_operar_como`	Inicio, fin y cada reserva o cancelación hechas durante "Operar como". Solo se inserta.

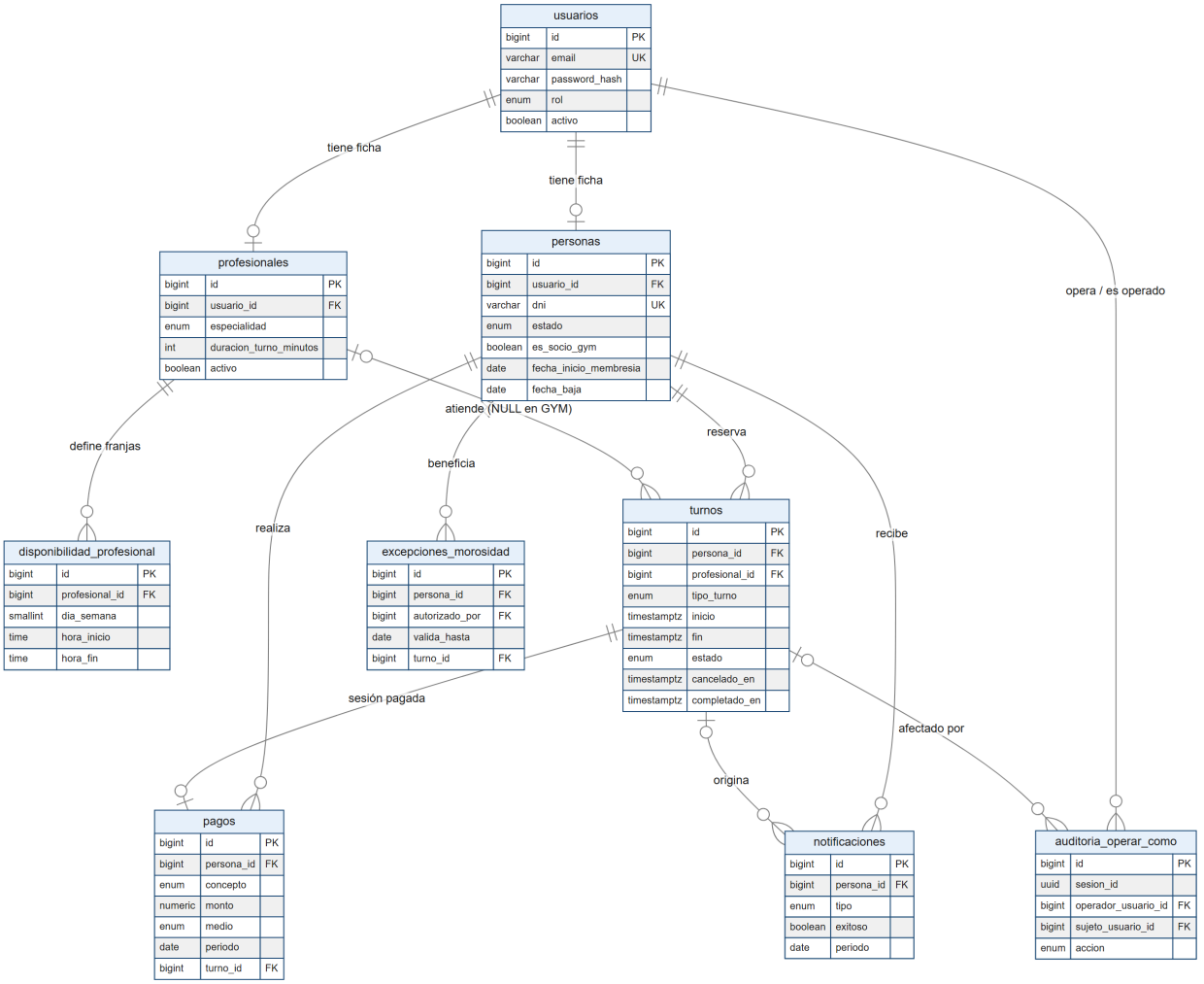


Figura 1. Modelo entidad-relación simplificado (atributos principales). El modelo completo está en `docs/02-diseno/diagrama-er.mermaid` y el detalle de cada columna en el diccionario de datos.

3.2. Reglas de negocio relevantes

Regla	Descripción	Dónde se garantiza
RN-01	**Morosidad del gimnasio.** Se recorren mes a mes los períodos desde el inicio de la membresía; cada período cuenta como impago si pasaron 10 días de su vencimiento sin cuota, aunque se hayan pagado meses posteriores. Con al menos un impago el socio queda "con deuda" pero conserva el acceso; al alcanzar el umbral configurable (6) queda suspendido y no puede reservar el gimnasio. Nunca bloquea un turno de consultorio.	Servicio (`MorosidadService`), compartido por la reserva y el estado de cuenta
RN-02 / RN-03	**Cancelación y ocupación del lugar.** El estado registra la anticipación del aviso (24 h en consultorio, 2 h en gimnasio). Lo que decide si el lugar se libera es el momento: toda cancelación anterior al inicio libera el lugar; una vez iniciada la franja, queda ocupado. La cancelación tardía no se cobra.	Base de datos (`fn_turno_ocupa_lugar`) y servicio

Regla	Descripción	Dónde se garantiza
RN-22	**Sin superposiciones.** Un profesional no puede tener dos turnos que se pisen, y una persona tampoco, aunque sean de distinto tipo.	Base de datos (restricciones `EXCLUDE USING GIST`)
RN-14 a RN-17	**Gimnasio.** Franjas de 60 minutos en punto (lunes a viernes de 7 a 21, sábados de 9 a 12), cupo por franja, un turno por persona y por día, solo socios activos y sin feriados.	Base de datos (trigger con bloqueo) y servicio
RN-18	**Anticipación de la reserva de gimnasio:** como máximo 7 días antes y como mínimo 1 hora antes.	Servicio (depende de la hora actual)
RN-05 a RN-07, RN-20	**Pagos.** Una cuota por persona y período; un pago por turno; la cuota lleva período y la sesión lleva turno; solo se paga una sesión de consultorio a nombre de la persona del turno.	Base de datos (índices únicos parciales, CHECK y trigger)
RN-08 / RN-19 / RN-28	**Asistencia.** El profesional marca atendido desde 15 minutos antes del inicio y ausente desde el inicio. En el gimnasio lo hace Recepción en el check-in, y si nadie lo marca el turno pasa a ausente al cerrar la franja. Se guarda la hora real del ingreso.	Servicio y tarea programada
RN-09 / RN-23	**Excepciones de morosidad** autorizadas por el ADMIN, de un solo uso o por período, solo para socios suspendidos.	Servicio y base de datos
RN-24 / RN-25	**Reducción de disponibilidad o baja del profesional:** los turnos futuros que quedan afuera se cancelan en cascada, con vista previa del impacto, confirmación explícita y aviso al paciente.	Servicio
RN-26 / RN-27	**Operar como:** credencial acotada y revocable; toda escritura queda auditada en la misma transacción.	Filtro de seguridad, servicio y base de datos

3.3. Decisiones de modelado importantes

- **Identidad única socio-paciente.** Una persona es una sola fila de `personas`, vinculada a un usuario, con su historial completo de cuotas y turnos. Es el núcleo de la propuesta de valor y el relevamiento lo confirmó como un caso frecuente.
- **Separación entre usuario y ficha.** `usuarios` guarda la autenticación; `personas` y `profesionales` guardan los datos del dominio. Esto permite que alguien se registre solo y que luego Recepción vincule su ficha por email, sin crear credenciales nuevas (RF-31).
- **Una sola tabla de turnos para dos circuitos distintos.** El turno de gimnasio no tiene profesional (`profesional_id` nulo) y se rige por cupo; el de consultorio depende de la disponibilidad del profesional. Compartir la tabla permite que una misma restricción impida que una persona tenga dos turnos superpuestos de distinto tipo, mientras que las validaciones propias de cada circuito siguen separadas.
- **Baja lógica en todo el modelo.** Personas y profesionales nunca se borran: el historial de pagos y turnos debe sobrevivir y una persona dada de baja suele volver.
- **Reglas críticas en el motor de base de datos.** Las restricciones de exclusión, los índices únicos parciales y los triggers son la defensa real: aunque el código tuviera un error, la base no acepta un estado inválido. La capa de servicio traduce esos rechazos a respuestas 409 o 422 con mensajes claros.
- **Tipos enumerados nativos de PostgreSQL** para roles, estados, especialidades, conceptos y medios de pago, de modo que un valor fuera de dominio sea imposible de guardar.
- **El período de una cuota es el primer día del mes**, controlado por una restricción, lo que hace inequívoca la unicidad de la cuota mensual.
- **Esquema versionado.** El esquema evolucionó con 17 migraciones de Flyway (V1 a V17) sin modificar las anteriores, lo que permite reconstruir la base desde cero en cualquier entorno.

3.4. Diagramas del dominio

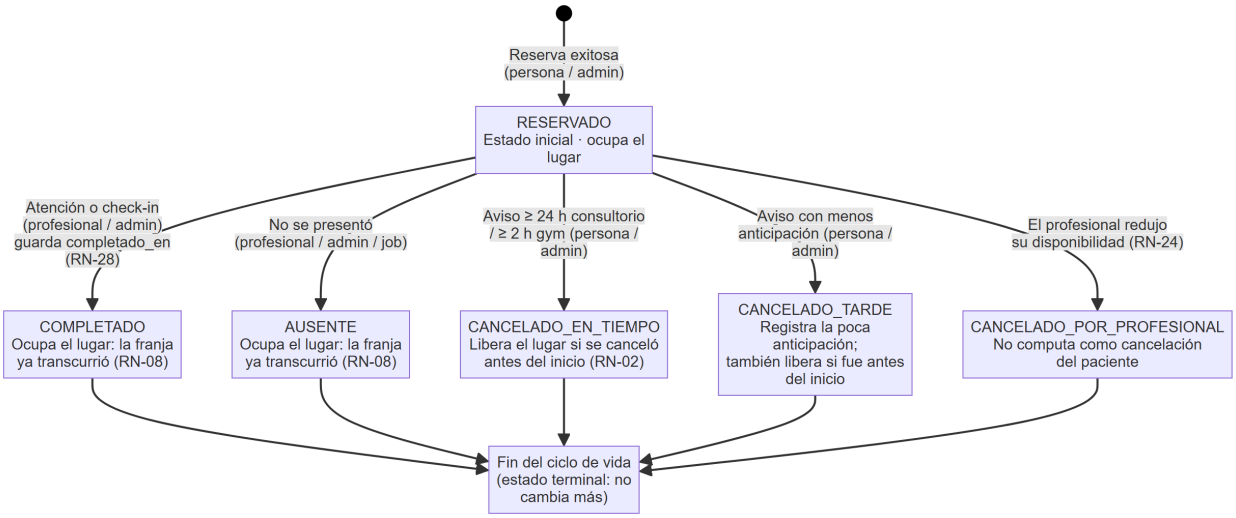


Figura 2. Diagrama de estados del turno: cada estado final indica si ocupa o libera el lugar.

El repositorio incluye además el diagrama de casos de uso, el de clases del dominio y los diagramas de secuencia de cada flujo principal (reserva de consultorio y de gimnasio, cancelación, reducción de disponibilidad, registro público, vínculo de fichas, excepciones de morosidad y "Operar como") en `docs/02-diseno/diagramas.md`.

4. Arquitectura y tecnologías

4.1. Arquitectura final

La solución sigue una arquitectura cliente-servidor de tres capas. El frontend es una aplicación de una sola página (SPA) que consume una API REST con JSON. El backend está organizado en capas (Controller → Service → Repository): los controladores exponen los endpoints y validan la entrada, los servicios concentran las reglas de negocio y el control de acceso por rol, y los repositorios acceden a la base con Spring Data JPA. La base de datos PostgreSQL es, además de almacenamiento, la última línea de defensa de las reglas críticas.

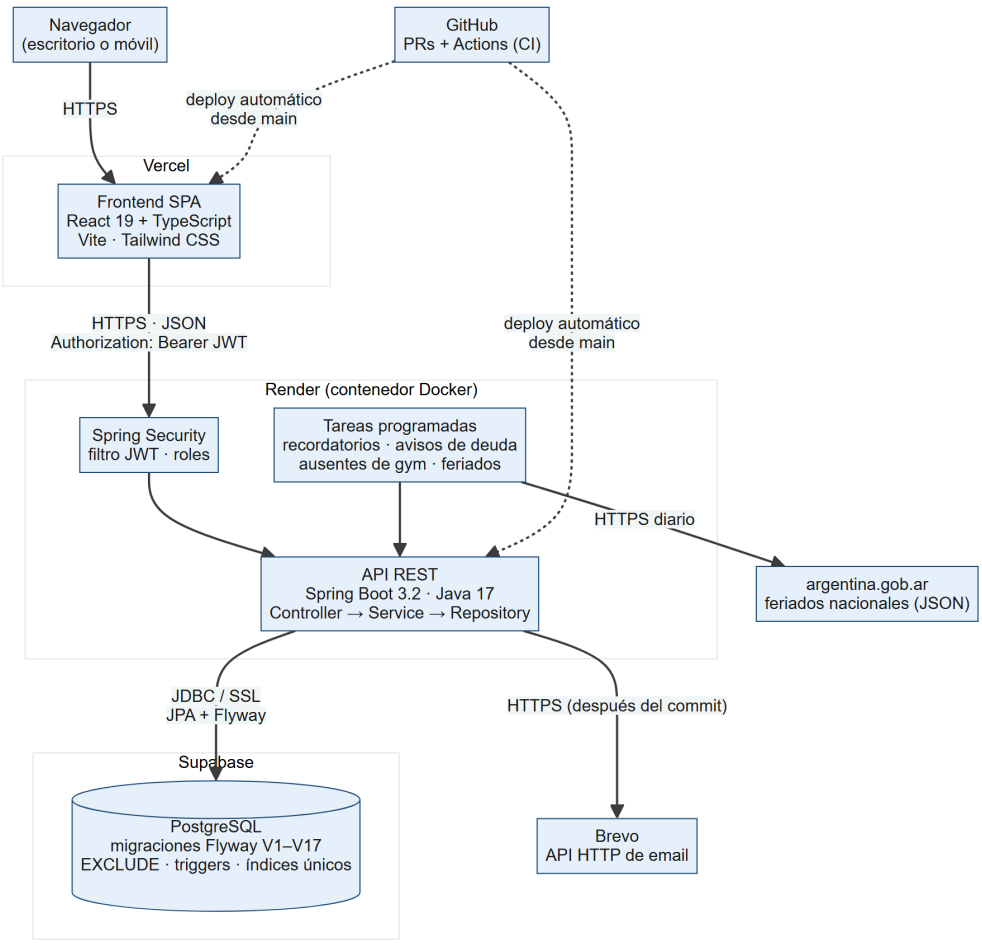


Figura 3. Arquitectura y despliegue.

4.2. Componentes, responsabilidades y motivo de cada tecnología

Componente	Tecnología	Responsabilidad	Por qué se eligió
Frontend	React 19 + TypeScript, Vite, React Router, Tailwind CSS	Interfaz por rol, navegación, formularios, estados de carga y error; nunca decide una regla de negocio.	Estándar de la industria, con el que el equipo ya trabajó; el tipado estático reduce errores al consumir la API.
Backend	Java 17, Spring Boot 3.2, Spring Web, Spring Data JPA, Bean Validation	API REST, reglas de negocio, transacciones, tareas programadas e integraciones.	Trabajado en profundidad en Programación III. Se priorizó una tecnología conocida para invertir el tiempo en el producto y no en aprender un framework con fecha de entrega fija.
Seguridad	Spring Security, JWT (jjwt), bcrypt	Autenticación sin estado, autorización por rol y la credencial acotada de "Operar como".	Integración nativa con Spring; JWT permite un backend sin sesión en el servidor, adecuado para un PaaS que reinicia el contenedor.

Componente	Tecnología	Responsabilidad	Por qué se eligió
Base de datos	PostgreSQL (Supabase) con Flyway	Persistencia, integridad referencial, exclusión de solapamientos, cupos y unicidad.	El dominio es fuertemente relacional y transaccional. Se comparó con MongoDB en el RFC-002: las garantías que se necesitaban (exclusión por rangos de tiempo, transacciones) son nativas en PostgreSQL.
Documentación de la API	springdoc-openapi (Swagger UI)	Contrato navegable de todos los endpoints.	Se genera desde el código, por lo que no se desactualiza.
Despliegue	Render (backend en Docker), Vercel (frontend), Supabase (base)	Servir la aplicación por HTTPS con despliegue automático desde la rama principal.	Planes gratuitos suficientes para la escala del problema y sin administración de servidores. Render no ofrece un entorno nativo de Java, por eso el backend se publica como imagen Docker, la misma que se usa en local.
Entorno local	Docker Compose	Levantar PostgreSQL y el backend con un comando y cargar los datos de prueba.	Evita instalar Java, Maven y PostgreSQL a mano y asegura el mismo entorno para ambos integrantes.
Calidad e integración continua	JUnit 5, Mockito, Testcontainers, JaCoCo, Vitest, Testing Library, GitHub Actions	Pruebas unitarias y de integración contra un PostgreSQL real, umbral de cobertura y verificaciones de la documentación en cada Pull Request.	Las reglas críticas viven en la base, así que solo una base real (Testcontainers) prueba que se cumplen.

4.3. Integraciones realizadas

- **Brevo (envío de correo por API HTTP).** Detrás de una interfaz `NotificadorEmail`, con una implementación que solo registra en el log para desarrollo y pruebas. Envía las confirmaciones, cancelaciones, recordatorios y avisos de deuda.
- **Calendario oficial de feriados.** Una tarea diaria descarga el JSON de feriados nacionales publicado en argentina.gob.ar y actualiza la tabla `feriados`; el gimnasio no abre los feriados inamovibles ni los trasladables en su fecha corrida. El ADMIN puede además cargar cierres propios y forzar la sincronización.
- **Supabase, Render y Vercel**, con despliegue continuo desde la rama principal de GitHub. El backend informa en `/api/health` qué commit está corriendo, lo que permite detectar un frontend nuevo hablando con un backend viejo.
- **GitHub Actions**, con tres trabajos: consistencia de la documentación (que incluye el control del prototipo exportado), backend (`mvn verify` con umbral de cobertura) y frontend (compilación y pruebas con umbral).

4.4. Seguridad, autenticación y autorización

- **Contraseñas** almacenadas con hash bcrypt, nunca en texto plano.
- **Autenticación con JWT** de 24 horas en el encabezado `Authorization`. Todos los endpoints, salvo registro, login, health y la documentación, requieren un token válido; un token vencido, malformado o con firma inválida responde 401.
- **Autorización por rol en la capa de servicio** (RNF-03), además de la regla declarativa por ruta: un socio solo ve sus propios turnos y su estado de cuenta, un profesional solo su agenda, y, fuera

de la administración, el DNI del paciente solo aparece en la agenda del profesional que lo atiende.

- **Registro público con rol forzado:** si el cuerpo de la solicitud trae un campo `rol`, se rechaza con 400 antes de crear nada.
- **Datos personales (Ley 25.326).** Que una persona tenga un turno de Psicología ya es un dato de salud. Se aplicó minimización (no se guarda contenido clínico), acceso por rol y transporte por HTTPS. Los datos de demostración son ficticios y los correos de prueba usan el dominio reservado `example.com`, que nunca entrega mensajes.
- **Secretos fuera del repositorio:** credenciales de base, clave JWT y API key de correo se cargan como variables de entorno en Render.
- **Correos seguros:** los nombres que se insertan en el HTML de los correos se escapan para evitar inyección de contenido.
- **CORS:** la API acepta solicitudes de cualquier origen. Es aceptable porque la autenticación viaja en un encabezado y no en cookies, de modo que un sitio ajeno no puede usar la sesión del usuario; restringirlo al dominio del frontend queda como mejora.

4.5. "Operar como": actuar en nombre de otra persona

Es la decisión de seguridad más sensible del proyecto, porque Recepción actúa en nombre de un socio. La Decisión 5 del RFC ya admitía que la administración reservara y cancelara por una persona; lo nuevo es la forma. En lugar de duplicar en endpoints de ADMIN las reglas del socio, el ADMIN obtiene una credencial cuyo sujeto es el socio y cuyo actor es el ADMIN (claim `act`, RFC 8693). Así el sistema le aplica exactamente las mismas reglas que al socio (morosidad, un turno por día, anticipación) y el backend sigue siendo la única autoridad. Los límites son:

- dura 30 minutos, no se renueva y no se puede anidar; solo se puede suplantar a un socio habilitado, nunca a un ADMIN ni a un profesional;
- en cada solicitud el filtro revalida al actor, al sujeto y que la sesión tenga un registro de inicio y no de fin; terminar la sesión la revoca en el servidor aunque el token no haya vencido;
- todo lo exclusivo del ADMIN responde 403 y solo se permiten cuatro escrituras (reservar gimnasio, reservar consultorio, cancelar y terminar la sesión);
- cada reserva o cancelación se audita en la misma transacción: si la auditoría falla, el turno no se guarda; el turno registra al ADMIN real como autor y el socio ve la marca "Reservado por Recepción".

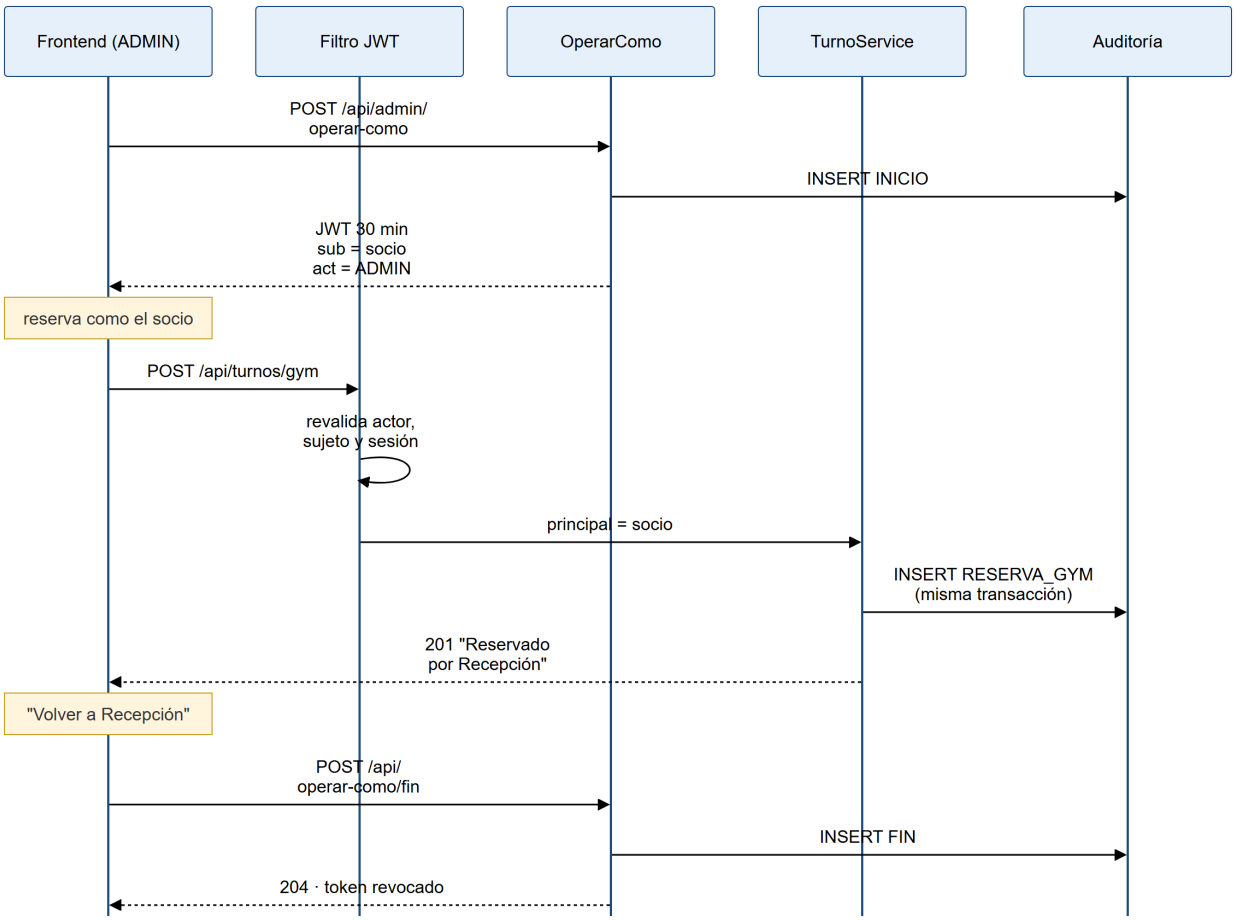


Figura 4. Secuencia de "Operar como": inicio, reserva auditada y fin de la sesión.

4.6. Concurrencia, transacciones y sincronización

- **Doble reserva del mismo horario.** Las restricciones `EXCLUDE USING GIST` sobre rangos de tiempo hacen imposible que dos turnos superpuestos queden confirmados: si dos personas reservan a la vez, la base acepta una y rechaza la otra (409).
- **Cupo del gimnasio.** Contar los turnos de una franja y luego insertar es una condición de carrera clásica: dos transacciones podían leer "19 de 20" y entrar ambas. El trigger toma un `pg_advisory_xact_lock` por franja, lo que serializa solo las reservas de esa misma franja sin bloquear al resto.
- **Unicidad bajo concurrencia** de cuotas, pagos por turno y avisos de deuda por período, garantizada con índices únicos parciales: el índice es la defensa real, no una consulta previa en el código.
- **Correos después del commit.** La notificación se guarda en la misma transacción que la reserva o la cancelación, y el envío ocurre recién después de confirmada, con un reintento. Así no se manda un correo de una reserva que luego se revirtió, y una caída del proveedor no impide reservar.
- **Tareas programadas:** recordatorios cada hora, avisos de deuda una vez por día, sincronización de feriados de madrugada y paso a ausente de los turnos de gimnasio sin check-in. Como el plan

gratuito de Render duerme el proceso, la tarea de ausentes procesa cada vez todos los turnos vencidos, de modo que al despertar se pone al día con los que quedaron pendientes.

- **Hora y zona horaria.** El backend usa un `Clock` inyectado en la zona de Buenos Aires; las reglas que dependen de "ahora" se prueban de forma determinista fijando el reloj.

5. Implementación

5.1. Metodología y organización del trabajo

Se trabajó con un modelo incremental y marco Scrum, en cinco sprints de dos semanas y una etapa de cierre, según la planificación elaborada en Metodología de Sistemas I. Ambos integrantes trabajaron sobre todo el stack: cada módulo tuvo un responsable primario que lo llevó de punta a punta (modelo de datos, endpoint e interfaz) y el otro revisó el Pull Request. Ningún cambio se integró sin revisión y sin la integración continua en verde. El seguimiento se hizo con Issues y Pull Requests en GitHub, y el estado real de cada tarea está en `docs/metodologia/plan-sprints.md`.

Sprint	Período	Resultado
Sprint 1	31/08 – 13/09	Esquema inicial, proyectos Spring Boot y React, despliegue en Render y Vercel.
Sprint 2	14/09 – 27/09	Autenticación y roles con JWT; segunda entrega (esquema de base de datos y módulos).
Sprint 3	28/09 – 11/10	Personas y Profesionales con disponibilidad; relevamiento de campo y ajustes de reglas.
Sprint 4	12/10 – 25/10	Agenda de turnos de gimnasio y de consultorio, excepciones de morosidad.
Sprint 5	26/10 – 08/11	Pagos, notificaciones, avisos de deuda, panel "Hoy en el centro", interfaz según el diseño y "Operar como".
Cierre	09/11 – 14/11	Estabilización, prueba sobre el despliegue, informe, video y preparación de la defensa.

Durante el desarrollo se utilizaron herramientas de inteligencia artificial como asistente técnico, conforme a los lineamientos de la cátedra: para revisar la consistencia del alcance, contrastar alternativas, explorar casos borde del dominio y asistir en la escritura de código y pruebas. Las decisiones de alcance, de diseño y de tecnología fueron analizadas y tomadas por el equipo, y todo el código pasó por revisión y por la integración continua.

5.2. Principales módulos desarrollados

Módulo	Backend (clases principales)	Frontend (pantallas)
Autenticación	`AuthService`, `JwtService`, `JwtAuthenticationFilter`, `SecurityConfig`	Login, Registro, Recuperar contraseña (informativa)
Operar como	`OperarComoService`, `OperarComoAutenticador`, `OperarComoInterceptor`, `AuditoriaOperarComoService`	Acción en Personas y banner persistente en el shell
Personas	`PersonaService`, `PersonaSpecifications`	Personas (ADMIN), con filtros, cuota y menú de acciones

Módulo	Backend (clases principales)	Frontend (pantallas)
Profesionales	`ProfesionalService`, `DisponibilidadService`, `CascadaDisponibilidadService`, `ImpactoCalculator`	Profesionales, Disponibilidad
Turnos	`TurnoService`, `MorosidadService`, `ExcepcionMorosidadService`, `JobAusentesGym`, `SincronizadorFeriados`	Inicio del socio, Reservar gimnasio, Turno de consultorio, Mis turnos, Agenda del profesional, Ingreso al gimnasio
Pagos	`PagoService`	Pagos (ADMIN) con modal de registro, Estado de cuenta (socio)
Notificaciones	`AvisosTurnoService`, `AvisosDeudaService`, `DespachadorNotificaciones`, `BrevoNotificadorEmail`, `JobRecordatorios`, `JobAvisosDeuda`	Notificaciones fallidas en "Hoy en el centro"
Panel del ADMIN	`ResumenHoyService`	Hoy en el centro

El frontend está organizado con diseño atómico (átomos, moléculas, organismos, plantillas y páginas), un cliente HTTP mínimo que adjunta el token, un contexto de autenticación, un contexto de tema y un guard de rutas por rol.

5.3. Recorridos y casos de uso principales

Reserva de un turno de gimnasio

El socio elige día y franja. El servicio valida la anticipación, consulta el veredicto de morosidad y, si el socio está suspendido sin excepción vigente, responde 422 con un mensaje que explica el motivo. Si puede reservar, inserta el turno; la base valida grilla, feriados, cupo, un turno por día y superposición. Con la reserva aceptada se registra la notificación en la misma transacción y el correo de confirmación se envía después del commit.

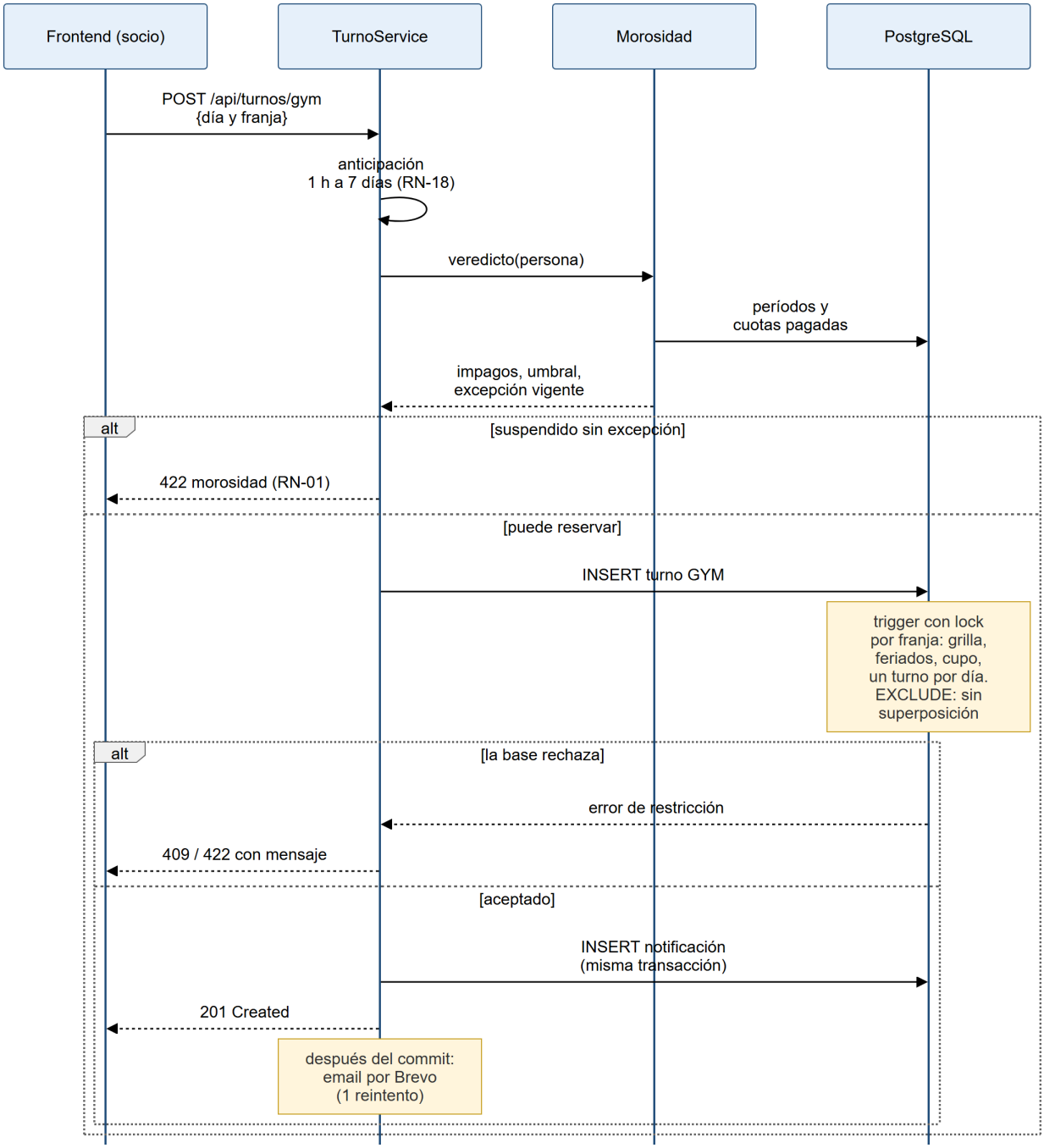


Figura 5. Secuencia de la reserva de un turno de gimnasio.

Otros recorridos

- **Turno de consultorio:** el socio elige especialidad y ve, por profesional, el primer día con turnos libres en los próximos 21 días; luego elige día y horario según la disponibilidad real. La deuda del gimnasio nunca bloquea este circuito.

- **Cancelación:** el socio o el ADMIN cancela con un motivo; el estado queda "en tiempo" o "tarde" según la anticipación y el lugar se libera si la franja no empezó. Se envía el aviso de cancelación.
- **Agenda del profesional:** ve sus turnos del día y de los próximos días, con los huecos libres y el turno en curso, y marca atendido o ausente dentro de la ventana permitida.
- **Reducción de disponibilidad:** antes de guardar, el sistema muestra qué turnos futuros quedarían afuera; al confirmar, los cancela con estado "cancelado por profesional" y avisa a cada paciente.
- **Check-in del gimnasio:** Recepción busca a la persona en la pantalla de Ingreso y registra la hora real de entrada.
- **Pagos:** el ADMIN busca a la persona, registra una cuota (el sistema sugiere el primer período impago) o una sesión de consultorio, con monto y medio; el estado de cuenta se actualiza al instante.
- **Avisos de deuda:** una tarea diaria detecta períodos impagos nuevos y envía un único aviso por período: de deuda o, si alcanzó el umbral, de suspensión.

5.4. Capturas de la aplicación

Las siguientes capturas se tomaron de la aplicación ejecutándose con los datos de demostración del repositorio.

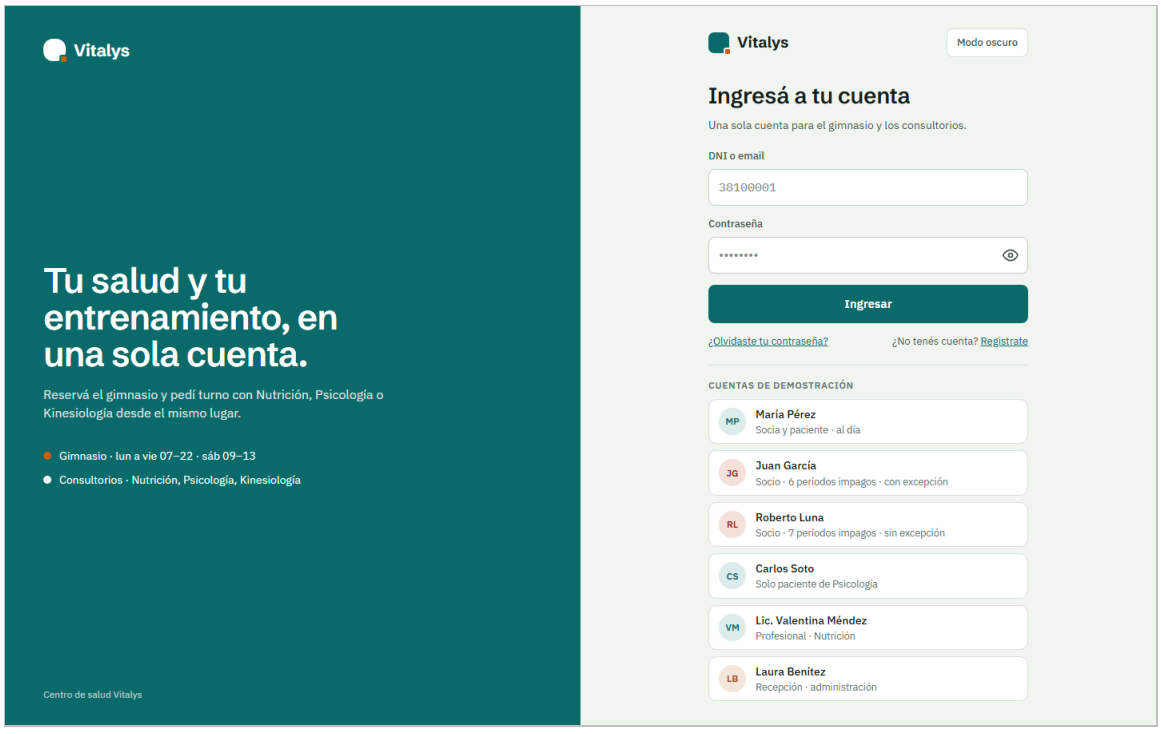


Figura 6. Ingreso con email o DNI y cuentas de demostración de los tres roles.

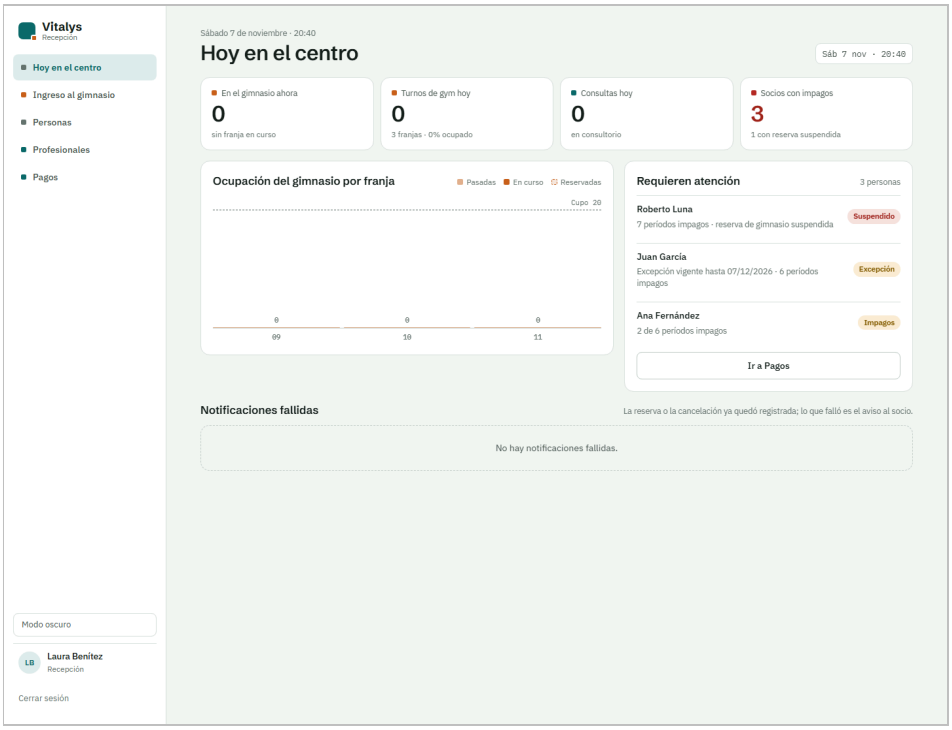


Figura 7. "Hoy en el centro", inicio del ADMIN: indicadores del día, ocupación del gimnasio por franja y socios que requieren atención por deuda.

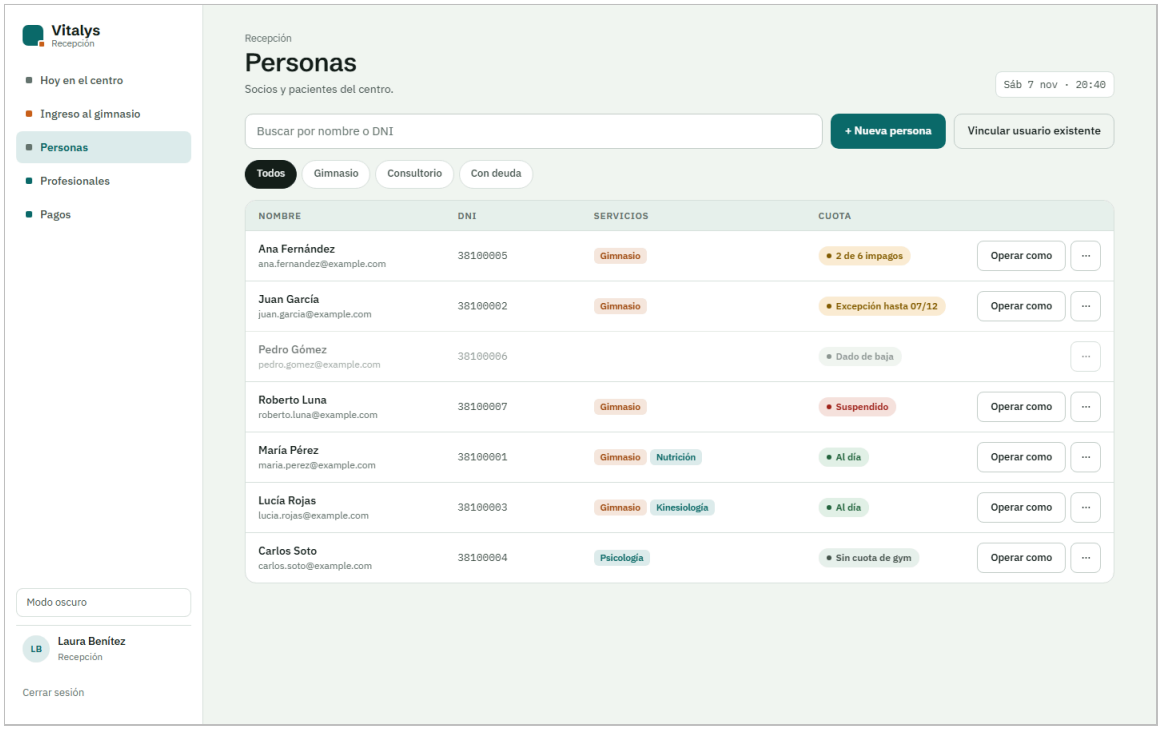


Figura 8. Personas: servicios que usa cada una, estado de la cuota y acción "Operar como".

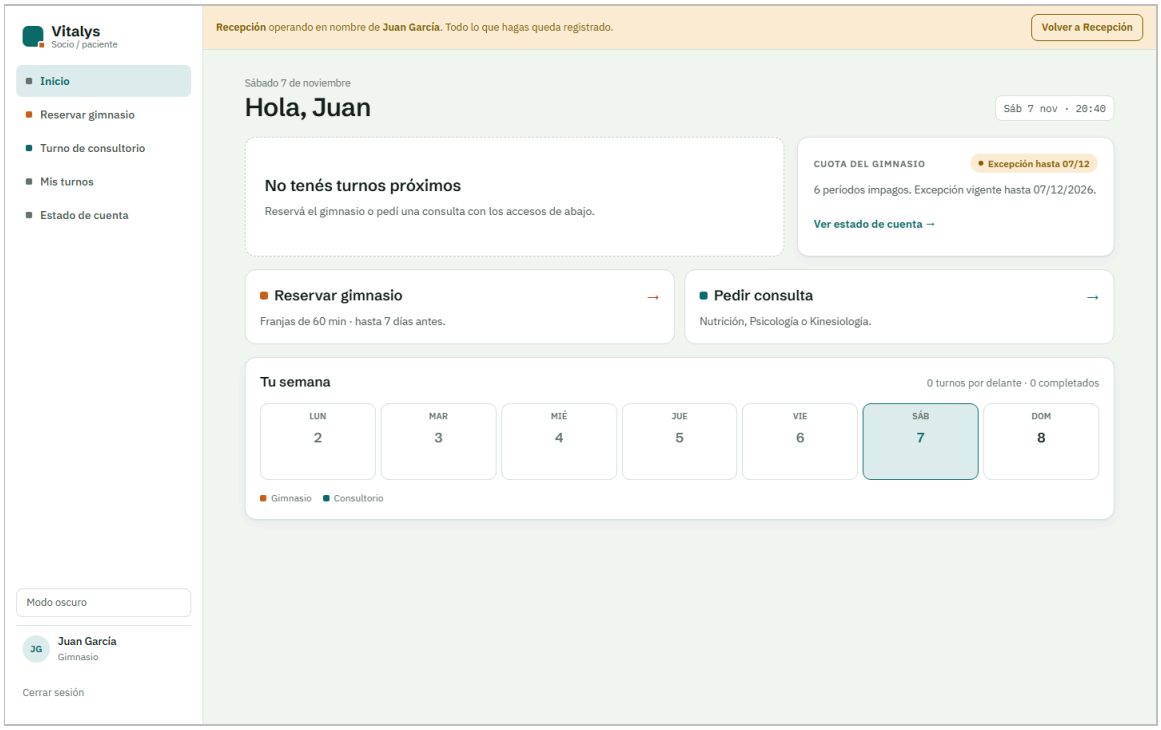


Figura 9. Recepción operando en nombre de un socio: banner persistente y botón "Volver a Recepción". El socio tiene una excepción de morosidad vigente.

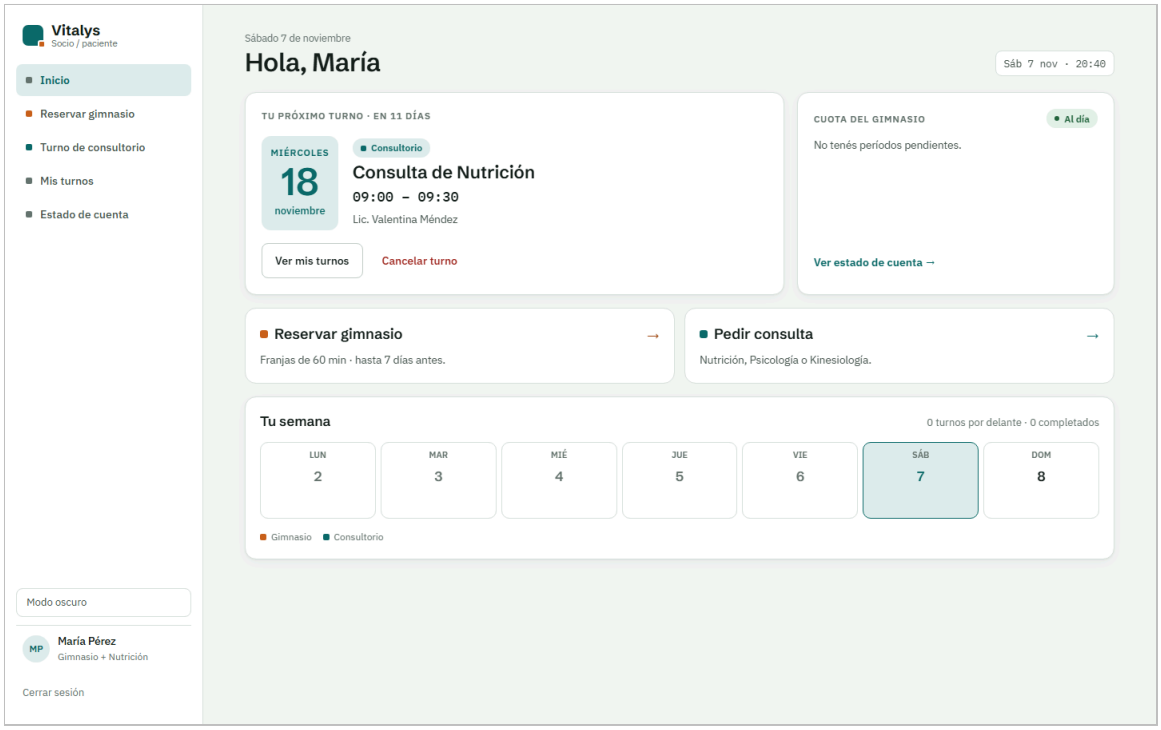


Figura 10. Inicio del socio: próximo turno, estado de la cuota y accesos a la reserva.

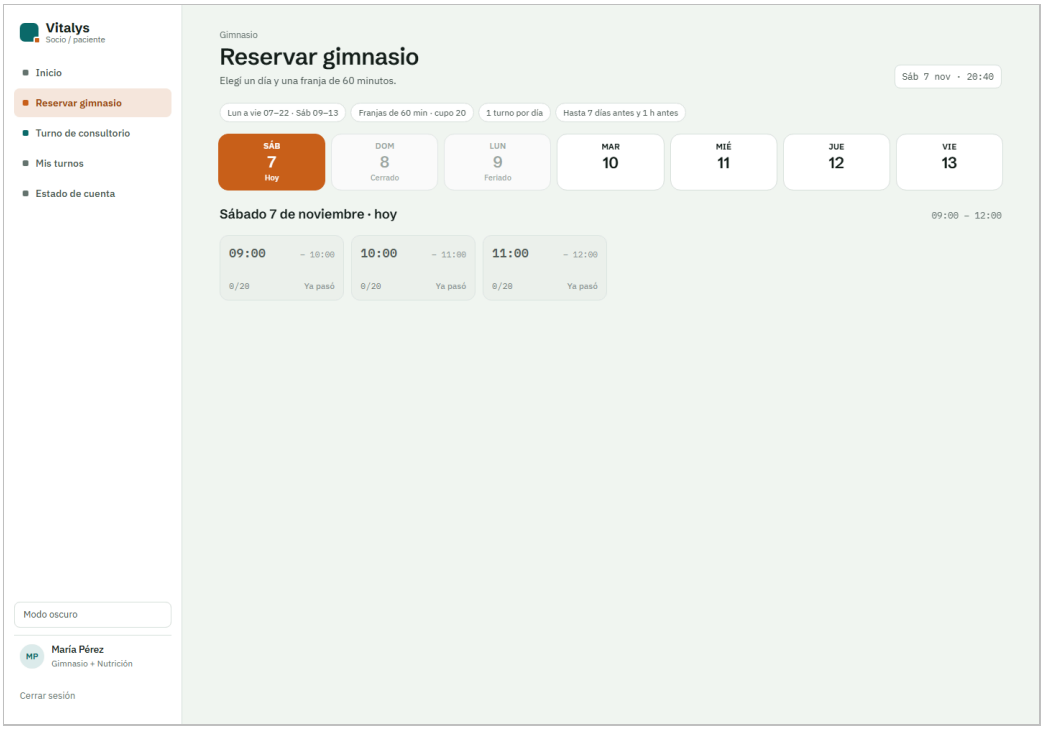


Figura 11. Reserva de gimnasio: días habilitados (hasta 7 días) y franjas con su cupo disponible.

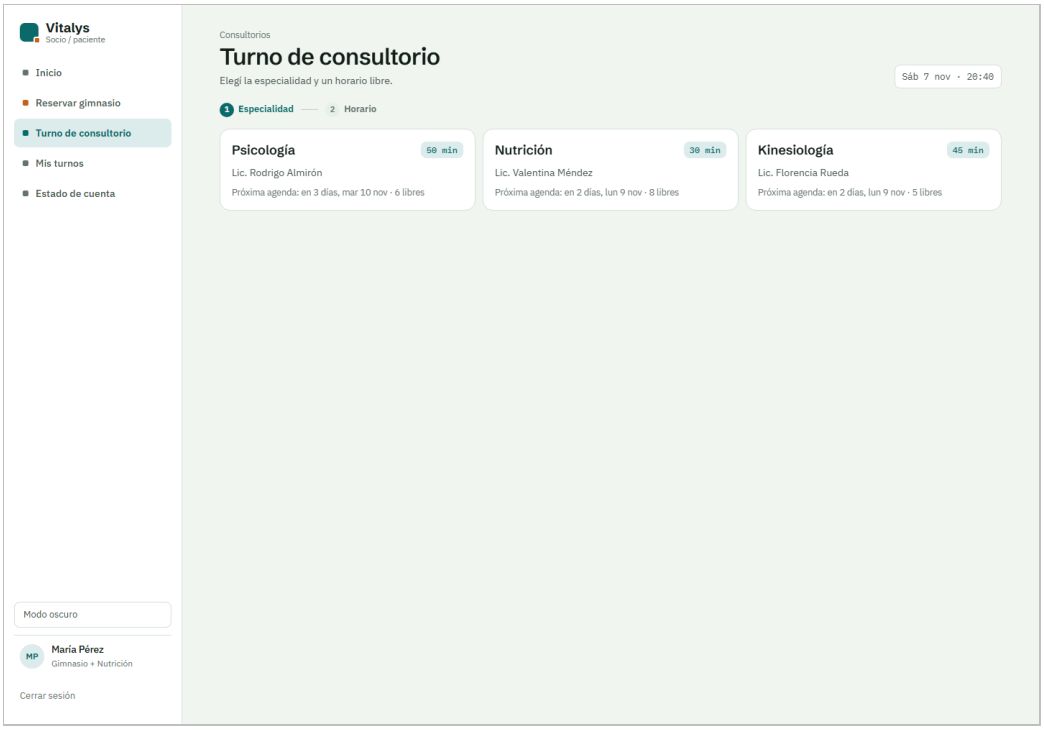


Figura 12. Turno de consultorio: profesionales por especialidad con su próxima agenda.



Figura 13. Estado de cuenta del socio: períodos pagados, medio de pago y explicación de la regla de suspensión.

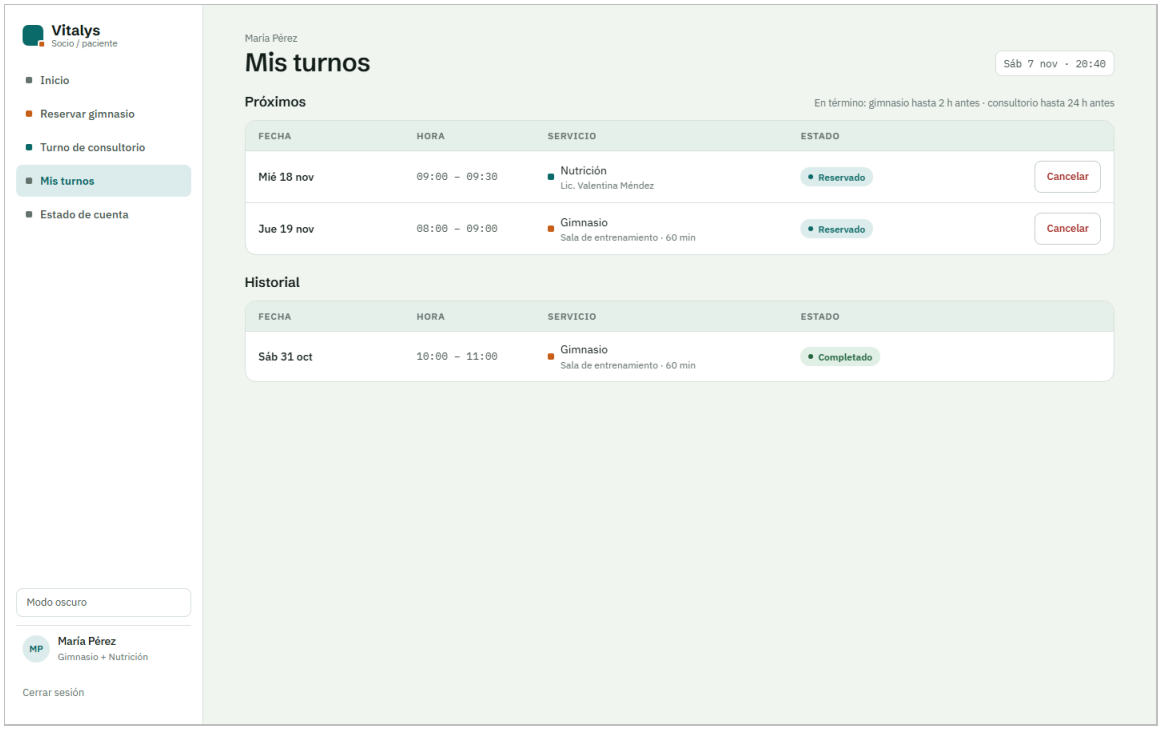


Figura 14. Mis turnos: próximos (con cancelación) e historial.

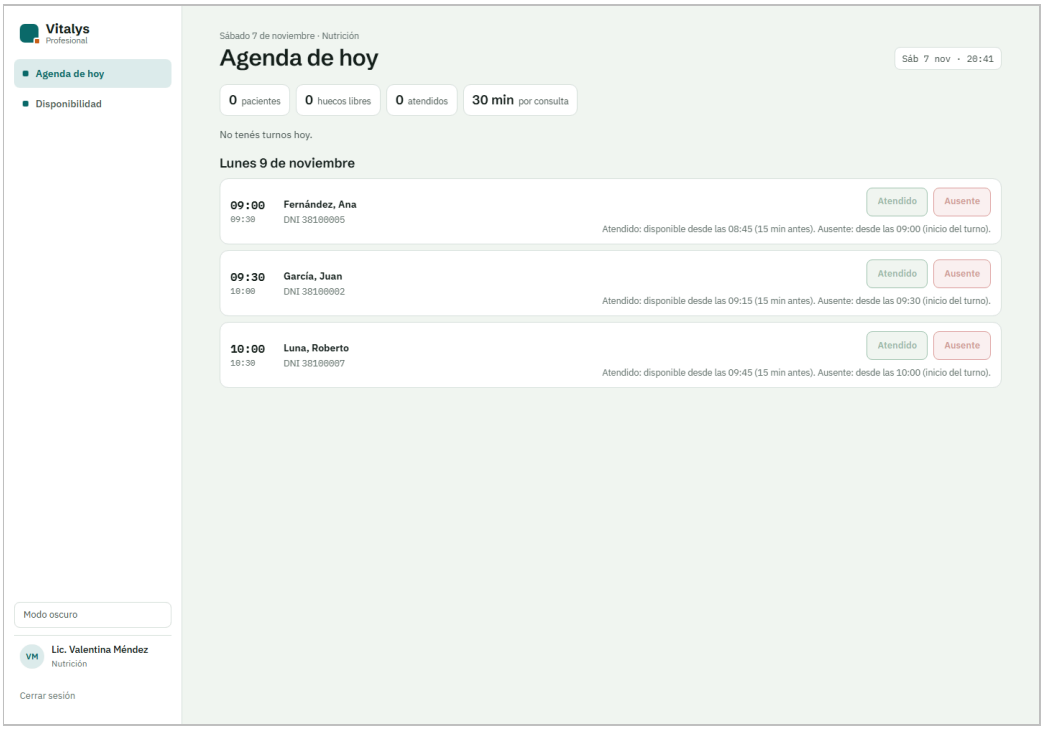


Figura 15. Agenda del profesional: turnos con el DNI del paciente y ventana habilitada para marcar atendido o ausente.

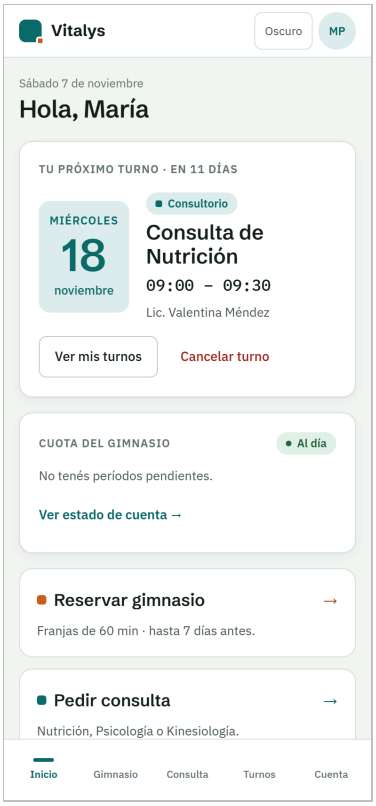


Figura 16. Vista en un celular: barra de pestañas inferior.

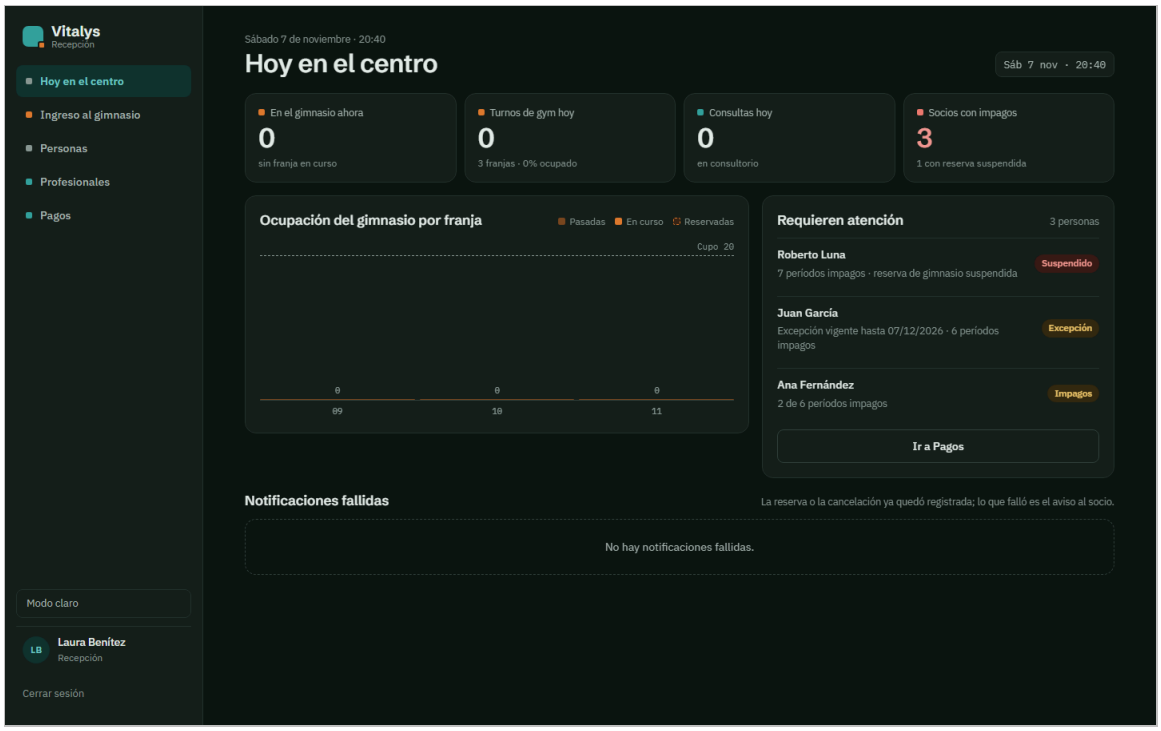


Figura 17. Modo oscuro persistente.

5.5. Dificultades técnicas relevantes y cómo se resolvieron

Dificultad	Solución
Hibernate envía los enumerados como texto y PostgreSQL rechaza ese parámetro contra sus tipos ENUM nativos, por lo que el mapeo JPA fallaba.	Mapeo con `@JdbcTypeCode(SqlTypes.NAMED_ENUM)`. Se registró como riesgo (RG-13) con su solución verificada.
Condición de carrera en el cupo del gimnasio: dos reservas simultáneas podían superar el cupo.	Bloqueo transaccional por franja (`pg_advisory_xact_lock`) dentro del trigger.
La regla de ocupación del lugar era inconsistente: la base retenía el lugar de una cancelación tardía en consultorio y lo liberaba en gimnasio.	Una única función `fn_turno_ocupa_lugar`, usada por los tres controles del motor (V8), alineada con el hallazgo del relevamiento.
Testcontainers no lograba comunicarse con Docker en Windows a través de los named pipes; más adelante, Docker 29 rompió la compatibilidad de la versión usada.	Ejecución de las pruebas de integración dentro de WSL, documentada en `setup-wsl-testcontainers.md`; luego, actualización de Testcontainers a 1.21.4.
Render no tiene un entorno nativo de Java, y el backend no arrancaba en producción ("Failed to configure a DataSource").	Publicación como imagen Docker, construida desde la raíz del repositorio para incluir las migraciones, y definición explícita del perfil `prod`, que era el que leía la URL de la base.
Tras un merge, el frontend nuevo consumió un backend todavía no redesplegado y la pantalla de Personas quedó en blanco.	El frontend tolera respuestas sin los campos nuevos y el backend informa en `/api/health` el commit desplegado, para detectar la desincronización.
Las tareas programadas no corren si Render durmió el proceso (RG-14).	La tarea de ausentes procesa todos los turnos vencidos cada vez que corre, así que al despertar se pone al día; además se precalienta el servicio antes de las demostraciones.

Dificultad	Solución
Enviar el correo dentro de la transacción podía notificar reservas que luego se revertían, o bloquear la reserva si el proveedor fallaba.	La notificación se persiste en la transacción y se envía después del commit, con un reintento y alerta al ADMIN si falla definitivamente.
Las reglas que dependen de la fecha actual hacían que las pruebas y los datos de demostración envejecieran.	Reloj inyectado en las pruebas y datos de demostración con fechas relativas al día de la carga.
Los socios del seed usaban un dominio de correo real, y con el proveedor activo podían recibir mensajes personas ajenas al proyecto.	Paso al dominio reservado `example.com`, que nunca entrega correos.

6. Pruebas y validación

6.1. Estrategia y qué aspectos se probaron

El plan de pruebas (`docs/metodologia/definicion-pruebas.md`) tiene tres niveles, y cada caso tiene un identificador que también figura en el nombre del método que lo prueba (por ejemplo, `pi09_…`), para que la trazabilidad entre documento y código sea directa:

1. **Unitarias** (JUnit 5 y Mockito en el backend; Vitest y Testing Library en el frontend): reglas de negocio aisladas, como el cálculo de morosidad con sus bordes, y el comportamiento de cada pantalla ante datos, errores y estados de carga.
2. **De integración** (Spring Boot Test y Testcontainers): la API completa contra un **PostgreSQL real** con todas las migraciones aplicadas. Es el único modo de comprobar que las restricciones del motor efectivamente rechazan lo que deben.
3. **Prueba de humo sobre el despliegue** (manual): los flujos principales de los tres roles contra Render, Vercel y Supabase, antes de cada entrega.

Los aspectos cubiertos son: autenticación y manejo de tokens inválidos; autorización por rol; ABM de personas y profesionales; disponibilidad y cascada; reservas, cancelaciones y estados de turnos; grilla, cupo y feriados del gimnasio; morosidad y excepciones; pagos y estado de cuenta; notificaciones y reintentos; "Operar como"; el panel del ADMIN; y la carga de los datos de demostración.

6.2. Evidencia de funcionamiento y resultados

Las pruebas se ejecutan en GitHub Actions en cada Pull Request, y el build falla si la cobertura de líneas baja del 90 % en el backend (JaCoCo) o en el frontend (Vitest). Ningún cambio se integró con la integración continua en rojo.

Indicador	Resultado
Backend: pruebas unitarias y de integración	426 unitarias y 201 de integración, todas en verde (CI de la rama principal tras el PR #39).
Frontend: pruebas	704 pruebas en 114 archivos, todas en verde, con 99,46 % de cobertura de líneas (ejecución sobre la rama principal del 07/11/2026).
Umbral de cobertura	90 % de líneas en ambos proyectos, con corte del build si no se alcanza.
Estado de la rama principal	Integración continua en verde en el último merge (PR #40, 07/11/2026).
Prueba de humo sobre el despliegue	Ejecutada el 27/10/2026 con los tres roles: todo el checklist pasó (ver 6.5).

6.3. Casos de prueba relevantes

ID	Caso	Resultado esperado y obtenido
PI-09	Dos reservas simultáneas del mismo horario de consultorio	Una responde 201 y la otra 409
PI-11 a PI-13	Cancelación en tiempo, tardía antes del inicio y con la franja ya iniciada	Las dos primeras liberan el lugar (otra persona lo reserva); la tercera lo mantiene ocupado (409)
PI-14 / PI-15	Cupo del gimnasio tras cancelaciones antes y después del inicio	Libera el cupo solo si se canceló antes del inicio
PI-10	Reserva de gimnasio de un socio suspendido sin excepción	422 con el motivo
PI-30 a PI-36, PI-81 a PI-83	Grilla horaria, cierre a la hora límite y feriados (inamovible, trasladable en su fecha corrida y no laborable)	422 fuera de grilla o en feriado; 201 en la última franja válida y en días no laborables
PU-01 a PU-05, PU-08 a PU-12	Morosidad: mes salteado con meses posteriores pagados, socio nuevo sin pagos, borde exacto del umbral, umbral leído de la configuración	Impagos y suspensión correctos en cada borde
PI-77 / PI-79	Segunda cuota del mismo mes; segundo pago del mismo turno	409 por los índices únicos
PI-93 / PI-96	Pago de sesión de un turno de gimnasio, de otra persona o cancelado	422: la cancelación tardía no genera cobro
PI-103	Estado de cuenta y estado de reserva del gimnasio	Coinciden siempre: es la misma regla
PI-18, PI-22 a PI-24	Registro con rol explícito; JWT vencido, malformado o con firma inválida	400 sin crear el usuario; 401
PI-117 / PI-119	Con la credencial de "Operar como", llamar a cada endpoint exclusivo del ADMIN (enumerados desde el mapeo real) o a datos de otro socio	403 en todos los casos
PI-120	Credencial vencida, actor o sujeto deshabilitado, credencial forjada sin inicio auditado	401
PI-121	Reserva suplantada cuando falla la auditoría	No queda el turno: la operación se revierte
PI-139	Reserva de gimnasio con menos de 1 hora o más de 7 días de anticipación	422; exactamente 1 hora y exactamente 7 días se aceptan

6.4. Cómo se comprueban las reglas críticas

- **Concurrencia.** PI-09 lanza dos reservas en paralelo desde dos hilos sincronizados con `CountDownLatch` contra un PostgreSQL real, y verifica que exactamente una gana. El cupo del gimnasio se protege con el bloqueo por franja descrito en 4.6.
- **Pagos.** La unicidad de la cuota y del pago por turno, el concepto excluyente y la imposibilidad de cobrar un turno cancelado se prueban contra las restricciones reales de la base, no contra validaciones del código.
- **Estados.** Cada transición del turno y su efecto sobre la ocupación del lugar (PI-11 a PI-15), y la ventana para marcar atendido o ausente, tienen casos propios.
- **Algoritmo de morosidad.** PU-01 a PU-05 y PU-08 a PU-12 recorren los bordes de la regla con fechas fijas mediante el reloj inyectado.

- **Seguridad.** Tokens inválidos, rol insuficiente, acceso a datos ajenos y todo el circuito de "Operar como" (PI-112 a PI-123).
- **Integraciones externas.** El correo se prueba con una implementación que registra los envíos, incluyendo la falla del proveedor, el reintento y la alerta en el panel; la sincronización de feriados se prueba con respuestas simuladas del calendario oficial, incluida su indisponibilidad.
- **Documentación consistente.** La integración continua verifica que los nombres de tablas, restricciones y triggers citados en la documentación existan en la base, que las fechas coincidan con su día de la semana y que las rutas citadas en las pruebas estén declaradas.

6.5. Validación sobre el despliegue y con el caso real

Prueba de humo sobre el despliegue (27/10/2026). Se ejecutó el checklist completo contra Render y Vercel con los tres roles: login de cada rol, alta de una persona, reserva y cancelación de un turno de consultorio (cancelado con menos de 24 horas, quedó "cancelado tarde" y el horario se liberó), registro de una cuota (la socia pasó de 2 a 1 período impago), reserva de un socio suspendido gracias a su excepción vigente, "Operar como" (403 en rutas de ADMIN, marca "Reservado por Recepción" y token revocado al terminar) y la regla de anticipación del gimnasio. Dos ítems no pudieron comprobarse con los datos de ese momento: marcar atendido un turno futuro responde 422, como corresponde, y debe verificarse el día del turno; y no había un socio suspendido sin excepción para ver el rechazo en vivo. Ese caso lo cubren las pruebas de integración y, desde entonces, los datos de demostración incluyen un socio en esa situación.

Validación con el caso real. La validación con el Centro Deportivo Jerárquicos fue sobre el dominio y las reglas, no sobre el uso del sistema: las reglas de morosidad, ocupación del lugar y ausencia del profesional se contrastaron con su operación real y se corrigieron en consecuencia (sección 1.3). **No se realizaron pruebas de usabilidad con usuarios del establecimiento**, por lo que la adecuación de la interfaz a usuarios reales queda como validación pendiente.

7. Resultados y conclusiones

7.1. Qué logró resolver el proyecto

- Cada persona existe una sola vez, con su historial de turnos de gimnasio y de consultorio y sus pagos en el mismo lugar.
- Las superposiciones de turnos y los cupos excedidos son imposibles por diseño, incluso con reservas simultáneas.
- La morosidad se calcula sola, con la tolerancia que usa un centro real, avisa antes de bloquear y nunca afecta la atención de salud.
- El socio reserva, cancela y consulta su cuenta en cualquier momento desde el celular; el profesional ve su agenda y registra la asistencia; Recepción tiene en una pantalla lo que requiere atención en el día y puede atender en el mostrador a quien no usa la aplicación.
- Toda operación crítica deja registro de quién la hizo y cuándo.

7.2. Qué cambió respecto de la situación inicial

Situación inicial	Con Vitalys
Socio y paciente registrados por separado, con datos inconsistentes.	Una identidad única con DNI único y un historial unificado.

Situación inicial	Con Vitalys
Agendas aisladas por profesional; superposiciones detectadas a mano.	Disponibilidad cargada por cada profesional y superposición rechazada por la base.
La deuda depende de que alguien la revise; el bloqueo es manual.	Cálculo automático, aviso por correo y suspensión al alcanzar el umbral, con excepciones registradas.
Reservar exige llamar en horario de atención.	Autoservicio las 24 horas; Recepción conserva el canal presencial con "Operar como".
Cancelaciones tardías que dejan franjas vacías.	Toda cancelación previa al inicio libera el lugar para otra persona.
Pagos sin trazabilidad.	Cada pago queda asociado a una persona, a un concepto, a un medio y a quien lo registró.

7.3. Limitaciones conocidas

- **Infraestructura gratuita:** el backend en Render se suspende sin tráfico (la primera solicitud puede tardar cerca de un minuto), Supabase pausa el proyecto tras unos siete días sin actividad y no hay copias de seguridad automáticas.
- **Cierre de sesión del lado del cliente:** el JWT de una sesión normal sigue siendo válido hasta su vencimiento de 24 horas (Decisión 9). La credencial de "Operar como", en cambio, sí se revoca en el servidor.
- **Sin recuperación de contraseña** por correo y **un solo rol por usuario**.
- **Cobro manual:** no hay débito automático ni pasarela de pago; tampoco pagos parciales.
- **Lecturas no auditadas** durante "Operar como": se auditan las escrituras, no las consultas.
- **CORS abierto** a cualquier origen (ver 4.4).
- **Sin validación de usabilidad** con usuarios reales del establecimiento.
- Las tareas programadas dependen de que el proceso esté activo: mientras Render lo tiene dormido, los turnos de gimnasio sin check-in siguen como reservados hasta que la tarea vuelve a correr.

7.4. Evolución futura

- Integración con una pasarela de pago y débito automático, que el modelo ya admite agregando un medio de pago.
- Recuperación de contraseña por correo y revocación de tokens en el servidor.
- Reportes de ingresos, ocupación y abandono para la dirección del centro.
- Actividades y clases grupales dentro del turno de gimnasio.
- Ficha clínica con el resguardo que exige la Ley 25.326, e integración de la rutina del gimnasio con la atención de salud (la necesidad que el propio establecimiento relevado señaló como pendiente).
- Varios roles por usuario, auditoría de lecturas en "Operar como", CORS restringido al dominio del frontend y resolución de ausentes al leer un turno vencido, sin depender de la tarea programada.
- Pruebas de usabilidad con socios, profesionales y personal de recepción.

7.5. Implementado y probado frente a lo proyectado

Implementado, desplegado y probado	Proyectado como trabajo futuro
Autenticación JWT con tres roles y control en la capa de servicio	Recuperación de contraseña; varios roles por usuario
Personas con identidad única, baja lógica y vínculo de usuario	Ficha clínica digital
Profesionales, disponibilidad y cancelación en cascada	—
Turnos de gimnasio y de consultorio con todas sus reglas, feriados oficiales y check-in	Actividades y clases grupales
Pagos manuales con medio de pago, estado de cuenta y morosidad con excepciones	Pasarela de pago, débito automático y pagos parciales
Notificaciones por correo con reintento y avisos de deuda	—
Panel "Hoy en el centro" y "Operar como" auditado	Reportes de gestión; auditoría de lecturas
Pruebas unitarias y de integración con umbral del 90 % y prueba de humo sobre el despliegue	Pruebas de usabilidad con usuarios reales

7.6. Aprendizajes y decisiones relevantes

- **El relevamiento cambia el diseño, no solo lo confirma.** Las dos reglas más importantes del sistema (la tolerancia de morosidad y la liberación del lugar al cancelar) se reescribieron a partir de lo que mostró un establecimiento real.
- **Poner las reglas críticas en la base de datos** resultó la decisión más rentable: la concurrencia y la integridad se resuelven donde realmente pueden garantizarse, y el código queda más simple.
- **Una prueba que pasa no siempre prueba algo.** Se encontró un caso que daba verde contra una ruta inexistente, porque el filtro rechazaba antes de llegar al controlador. Desde entonces la integración continua verifica que las rutas citadas en las pruebas existan.
- **La documentación también se prueba.** Los scripts que contrastan los nombres citados con el catálogo real de PostgreSQL detectaron inconsistencias que una lectura no encontraba.
- **Los cambios de alcance se registran.** Llevar cada cambio al RFC mediante Pull Request mantuvo alineados al equipo y a la tutora, y obliga a justificar decisiones sensibles como "Operar como".
- **Trabajar ambos sobre todo el stack**, con revisión cruzada de cada Pull Request, evitó que el conocimiento quedara concentrado en una persona y prepara a los dos integrantes para responder por todo el proyecto en la defensa.

Anexo. Documentación de referencia en el repositorio

Documento	Contenido
`docs/01-propuesta/propuesta-proyecto-rfc.md`	RFC-001: propuesta, relevamiento, decisiones de dominio y cambios de alcance
`docs/01-propuesta/relevamiento/`	Guía de entrevista, registro del relevamiento de campo y perfiles sintéticos
`docs/02-diseno/rfc-002-motor-base-de-datos.md`	Comparación PostgreSQL / MongoDB y decisión
`docs/02-diseno/requerimientos.md`	Requerimientos funcionales y no funcionales, reglas de negocio, historias de usuario y criterios de aceptación

Documento	Contenido
docs/02-diseno/modulos.md	Módulos y endpoints con sus permisos
docs/02-diseno/diccionario-datos.md y diagrama-er.mermaid	Modelo de datos completo
docs/02-diseno/diagramas.md	Casos de uso, estados, secuencias, clases y arquitectura
docs/02-diseno/wireframes.md y design-system/	Pantallas, prototipo y desvíos documentados
docs/03-despliegue/	Despliegue en la nube, entorno local con Docker y pruebas en Windows
docs/metodologia/	Plan de sprints con el estado real, matriz de riesgos y definición de pruebas
db/migration/	Migraciones V1 a V17 del esquema